# Lesson 7 Worksheet: The Challenge of Turning — ANSWER KEY

## Part 1: Heading from Coordinates

| Current Position | Next Position | row_diff | col_diff | Heading (number and name) |
|---|---|---|---|---|
| (0, 0) | (1, 0) | 1 - 0 = 1 | 0 - 0 = 0 | 2 (S) |
| (2, 3) | (1, 3) | 1 - 2 = -1 | 3 - 3 = 0 | 0 (N) |
| (1, 1) | (1, 2) | 0 | 1 | 1 (E) |
| (3, 2) | (3, 1) | 0 | -1 | 3 (W) |
| (0, 3) | (1, 3) | 1 | 0 | 2 (S) |
| (2, 0) | (2, 1) | 0 | 1 | 1 (E) |
| (3, 1) | (2, 1) | -1 | 0 | 0 (N) |
| (0, 2) | (0, 1) | 0 | -1 | 3 (W) |

## Part 2: The Right Turns Table

| Current  Needed | 0 (N) | 1 (E) | 2 (S) | 3 (W) |
|---|---|---|---|---|
| 0 (N) | 0 | 1 | 2 | 3 |
| 1 (E) | 3 | 0 | 1 | 2 |
| 2 (S) | 2 | 3 | 0 | 1 |
| 3 (W) | 1 | 2 | 3 | 0 |

- How many cells have 0 right turns? 4 — Why? When the current heading equals the needed heading, there are zero clockwise steps. These fall along the diagonal.
- What does 2 right turns mean physically? A 180-degree turn (two right turns = turning completely around).

- Do you notice a pattern in the table? Each row is the same sequence [0, 1, 2, 3] shifted. The values only depend on the distance between the headings going clockwise, not the headings themselves.

## Part 3: Determine the Number of Right Turns

| Scenario | Current Heading | Needed Heading | Clockwise Steps | Right Turns |
|---|---|---|---|---|
| 1 | 0 (N) | 1 (E) | 0->1 = 1 | 1 |
| 2 | 1 (E) | 1 (E) | already there = 0 | 0 |
| 3 | 2 (S) | 0 (N) | 2->3->0 = 2 | 2 |
| 4 | 3 (W) | 2 (S) | 3->0->1->2 = 3 | 3 |
| 5 | 1 (E) | 0 (N) | 1->2->3->0 = 3 | 3 |
| 6 | 0 (N) | 3 (W) | 0->1->2->3 = 3 | 3 |
| 7 | 2 (S) | 3 (W) | 2->3 = 1 | 1 |
| 8 | 3 (W) | 1 (E) | 3->0->1 = 2 | 2 |

- Why does counting clockwise always give the correct number of right turns? Each clockwise step corresponds to one right turn. Since the headings are arranged in a circle (0->1->2->3->0), counting steps clockwise from current to needed always gives the number of right turns needed. For example, going from 3 (W) to 1 (E): count 3->0->1, that is 2 steps, so 2 right turns. This works even when you have to "wrap around" past 0.

## Part 4: Trace a Path with Heading Updates

Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3)], Starting heading: 0 (N)

| Step | From | To | Needed Heading | Current Heading | Right Turns | New Heading |
|---|---|---|---|---|---|---|
| 1 | (0, 0) | (1, 0) | 2 (S) | 0 (N) | count 0->1->2 = 2 | 2 (S) |
| 2 | (1, 0) | (2, 0) | 2 (S) | 2 (S) | already there = 0 | 2 (S) |

| Step | From | To | Needed Heading | Current Heading | Right Turns | New Heading |
|---|---|---|---|---|---|---|
| 3 | (2, 0) | (2, 1) | 1 (E) | 2 (S) | count 2->3->0->1 = 3 | 1 (E) |
| 4 | (2, 1) | (2, 2) | 1 (E) | 1 (E) | already there = 0 | 1 (E) |
| 5 | (2, 2) | (2, 3) | 1 (E) | 1 (E) | already there = 0 | 1 (E) |

- How many steps required turning? 2 (step 1 with 2 right turns and step 3 with 3 right turns)
- How many steps required zero turns? 3 (steps 2, 4, 5)
- At which step did the robot change from row to column movement? Step 3

## Part 5: Trace a Second Path

Path: [(2, 3), (1, 3), (0, 3), (0, 2), (0, 1), (0, 0)], Starting heading: 2 (S)

| Step | From | To | Needed Heading | Current Heading | Right Turns | New Heading |
|---|---|---|---|---|---|---|
| 1 | (2, 3) | (1, 3) | 0 (N) | 2 (S) | count 2->3->0 = 2 | 0 (N) |
| 2 | (1, 3) | (0, 3) | 0 (N) | 0 (N) | already there = 0 | 0 (N) |
| 3 | (0, 3) | (0, 2) | 3 (W) | 0 (N) | count 0->1->2->3 = 3 | 3 (W) |
| 4 | (0, 2) | (0, 1) | 3 (W) | 3 (W) | already there = 0 | 3 (W) |
| 5 | (0, 1) | (0, 0) | 3 (W) | 3 (W) | already there = 0 | 3 (W) |

Final heading after completing path: 3 (W)

## Part 6: Trace a Third Path

Path: [(1, 1), (1, 2), (1, 3), (2, 3), (3, 3)], Starting heading: 0 (N)

| Step | From | To | Needed Heading | Current Heading | Right Turns | New Heading |
|---|---|---|---|---|---|---|
| 1 | (1, 1) | (1, 2) | 1 (E) | 0 (N) | count 0->1 = 1 | 1 (E) |
| 2 | (1, 2) | (1, 3) | 1 (E) | 1 (E) | already there = 0 | 1 (E) |
| 3 | (1, 3) | (2, 3) | 2 (S) | 1 (E) | count 1->2 = 1 | 2 (S) |
| 4 | (2, 3) | (3, 3) | 2 (S) | 2 (S) | already there = 0 | 2 (S) |

- Final heading: 2 (S)
- Does the first step require turning? Yes — Why? The robot starts facing 0 (North) but needs heading 1 (East), so it must make 1 right turn: count clockwise 0->1 = 1 step.

## Part 7: Robot Orientation Diagrams

- Scenario A: Facing 0 (N), need heading 1 (E) → Right turns needed: 1 — count 0->1 = 1
- Scenario B: Facing 1 (E), need heading 2 (S) → Right turns needed: 1 — count 1->2 = 1
- Scenario C: Facing 3 (W), need heading 1 (E) → Right turns needed: 2 — count 3->0->1 = 2
- Scenario D: Facing 2 (S), need heading 2 (S) → Right turns needed: 0 — already there

## Part 8: Heading Logic in Code

```
HEADING_NAMES = ["N", "E", "S", "W"]

def get_needed_heading(current_pos, next_pos):
    row_diff = next_pos[0] - current_pos[0]
    col_diff = next_pos[1] - current_pos[1]

    if row_diff == -1:
        return 0      # North
    elif col_diff == 1:
        return 1      # East
    elif row_diff == 1:
```

```
        return 2       # South
    elif col_diff == -1:
        return 3       # West
```

Test your understanding:

1. `get_needed_heading((0,0), (1,0))` returns **2** — row_diff = 1, so heading is 2 (South)
2. `get_needed_heading((2,3), (2,2))` returns **3** — col_diff = -1, so heading is 3 (West)
3. `get_needed_heading((1,1), (0,1))` returns **0** — row_diff = -1, so heading is 0 (North)

---

## Reflection

**What makes numeric headings and clockwise counting so powerful for solving the turning problem?** Using numbers 0-3 arranged clockwise means we can count steps from any heading to any other heading using a simple while loop. The loop walks clockwise one step at a time (adding 1 and wrapping from 3 back to 0), counting as it goes. This works for all 16 combinations automatically. It directly mirrors what the robot does physically - each step in the count is one right turn. The approach is easy to understand, easy to code, and reveals the underlying structure: headings are arranged in a circle, and counting clockwise naturally wraps around.